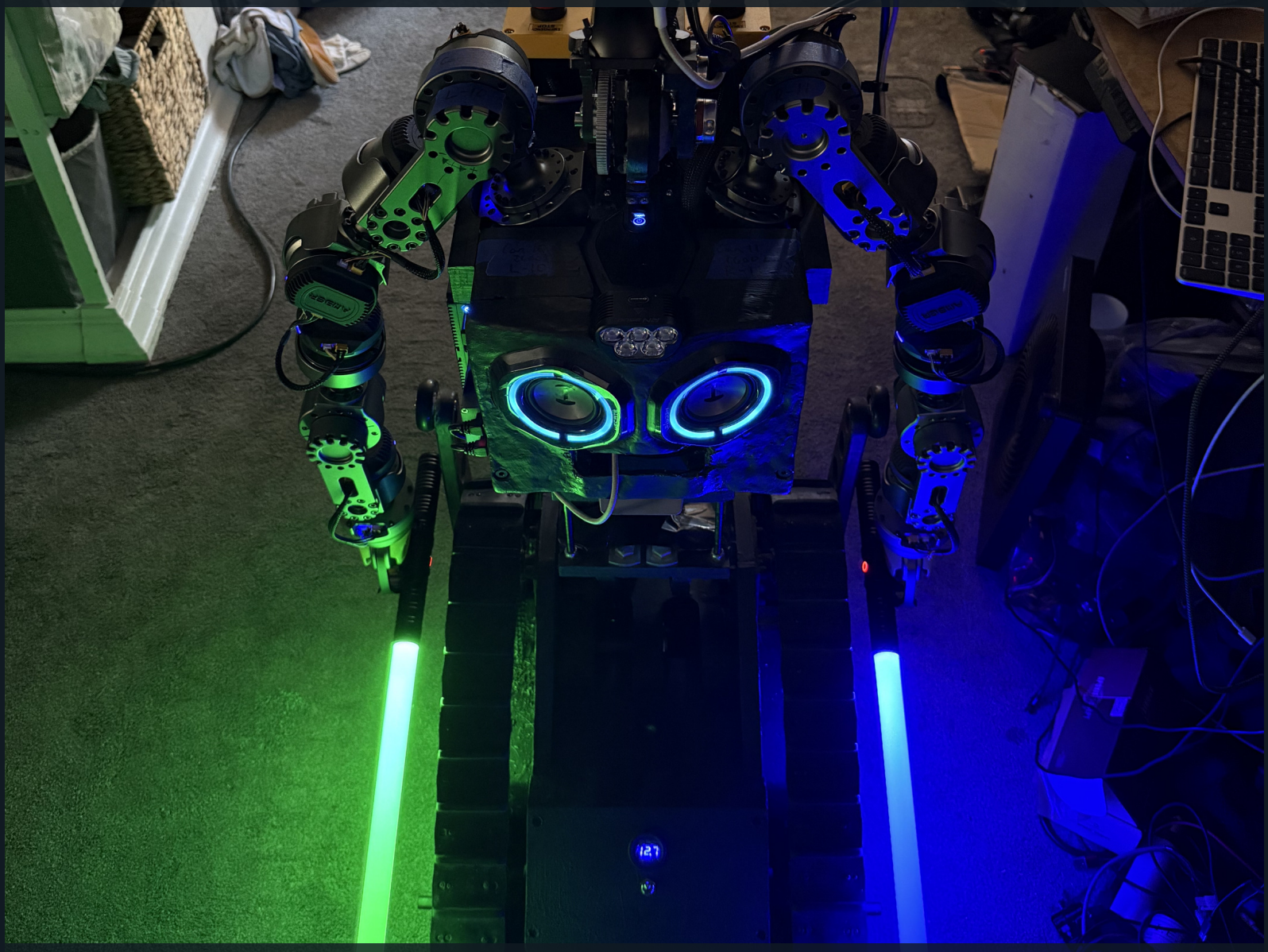
BUILDING R.O.B. - VOLUME 4

MISSION CONTROL

Arduino, Mac, networks, cameras, and responsible AI



A COMPUTER-SCIENCE FIELD GUIDE FOR MAKERS AGES 12-16

Rodolfo Aramayo / Orbitus Robotics - First Maker Faire Edition

Programs cooperate by making promises

Follow one command from a human hand to a moving tread, then follow sensor data back.

ROB's software spans Arduino C++, Objective-C and Objective-C++, Swift, Python helpers, Apple platforms, cameras, lidar, speech, and optional cloud AI. You do not need to master all of those at once. You need to understand the boundary each part owns.

This book describes the source snapshot inspected on 2 August 2026. Some Cerebro Gemini and autonomy files have uncommitted work, so experimental features are labeled. The labs stay in simulation unless a qualified adult provides an isolated test rig.

First evidence-based draft: 2 August 2026. Build details marked **Builder input needed** are deliberate placeholders for the builder to complete.

The Building R.O.B. library

VOLUME 1	*Meet ROB: A Robot Is a Team of Systems* - ages 8-12
VOLUME 2	*Circuits and Signals with ROB* - ages 10-14
VOLUME 3	*Motion Workshop: Treads, Gears, and Fabrication* - ages 10-15
VOLUME 4	*Mission Control: Arduino, Mac, Networks, and Vision* - ages 12-16
VOLUME 5	*AI, Robotics, and Codex: Directing, Verifying, and Owning AI-Built Systems* - advanced makers
VOLUME 6	*Dual-Arm Robotics: AMBER B1, URDF, CAN, and Ubuntu* - advanced makers
VOLUME 7	*Engineering ROBControllerVision: Swift, Spatial Input, Video, and Speech* - Swift developers
MANUAL	*Building R.O.B.: The Complete Maker's Field Manual* - advanced builders

HOW TO USE THIS BOOK

Use the diagrams to predict where data travels, where it is validated, and which component owns the final safety decision. Then run the paper or simulator mission. Never test new network or AI code first on powered treads.

ONE CURRENT ARDUINO, THREE HISTORICAL SKETCHES

The `ROBArduino` archive preserves Base, Torso, and Head sketches from an earlier three-Arduino design. The builder reports that present-day ROB uses **only the Base sketch**. This book therefore treats Base behavior as the current low-level firmware reference and labels Torso and Head behavior as retired historical evidence. Source files cannot prove which binary is flashed today; record the board identity, firmware hash, build environment, and upload date before operation.

TWO AMBER B1 ARMS, ONE DOCUMENTED CONTROL CHAIN

ROB carries two seven-joint AMBER B1 arms. The repository snapshot separates their Ubuntu-side cores as `L-10` and `R-11`: network clients use UDP or LCM, each core binds a stable CAN interface, and CAN carries commands and feedback between that core and its arm actuators. The `AmberHomeFolder` tree is evidence from one Ubuntu machine, not a deployable image; never copy its credentials, logs, virtual environment, or machine-specific identifiers. Volume 6 documents the inspected protocol, URDF models, reproducible setup, verification gates, and unresolved configuration differences.

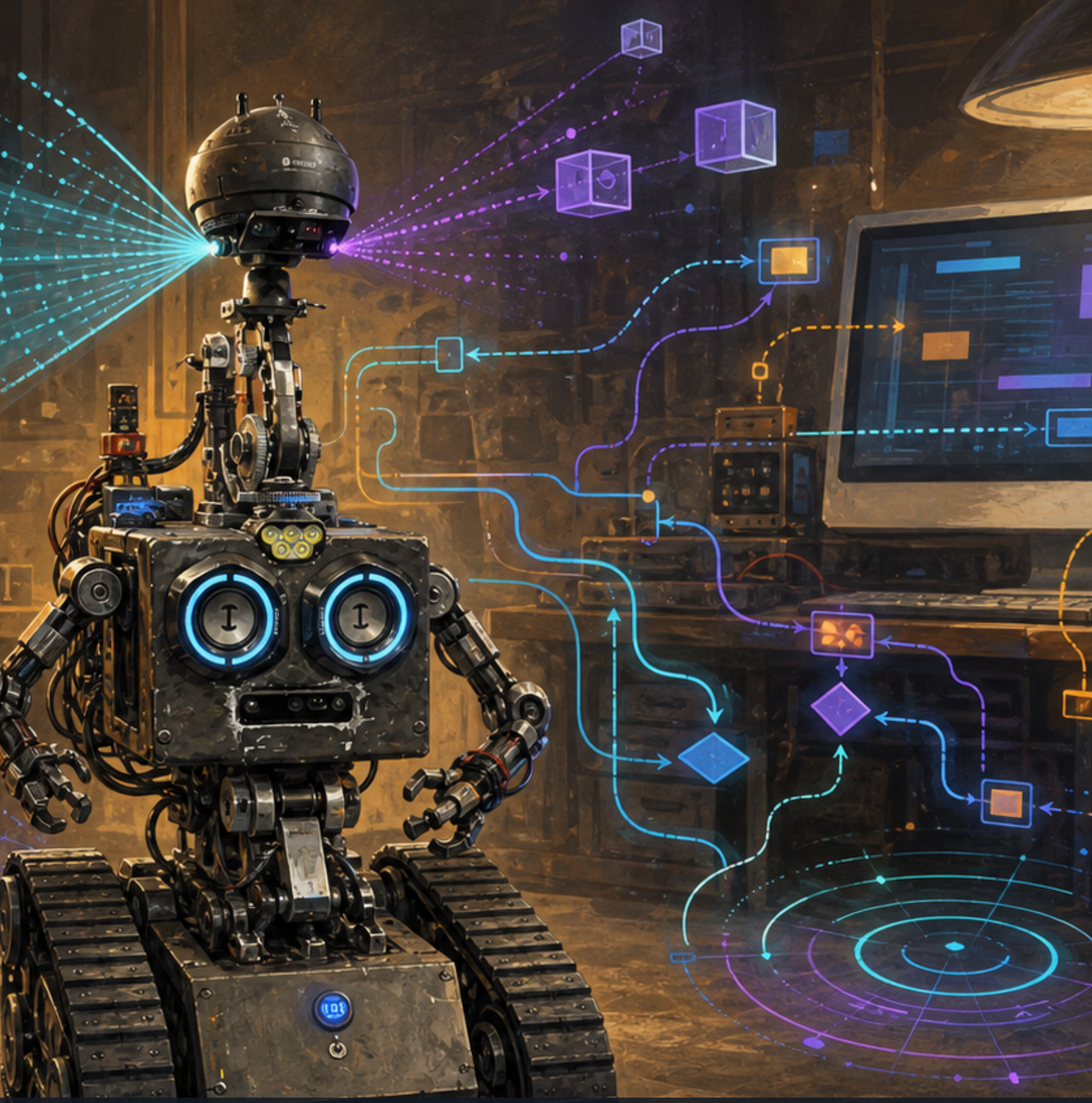
HOW TO FIND THE FILES

Open the public repositories on GitHub and follow each printed repository-relative path: `https://github.com/RudyAramayo/ROBBooks`, `https://github.com/RudyAramayo/ROBArduino`, `https://github.com/RudyAramayo/Cerebro`, `https://github.com/RudyAramayo/ROBController`, `https://github.com/RudyAramayo/ROBControllerVision`, `https://github.com/RudyAramayo/Amber-HomeFolder`, and `https://github.com/RudyAramayo/ROBTrainingGames`. The website is published separately at `https://github.com/OrbitusRoboticsWebSite/ORobotics`. See `https://github.com/RudyAramayo/ROBBooks/blob/main/ROB-Books/OPEN-SOURCE-CODE-MAP.md` for clickable repository and file links, license cautions, and renamed-file search instructions. A named archival item without a verified public repository is labeled as evidence, not given an invented URL.

SOFTWARE CAN MOVE HARDWARE

A one-line program can command a high-energy actuator. All exercises default to printed frames, LEDs, or a simulated robot. Connecting to ROB requires a named operator, an exclusion zone, a spotter, a tested physical emergency stop, lifted-tread sign checks, strict speed limits, and permission from the builder.

Contents



MISSION 1

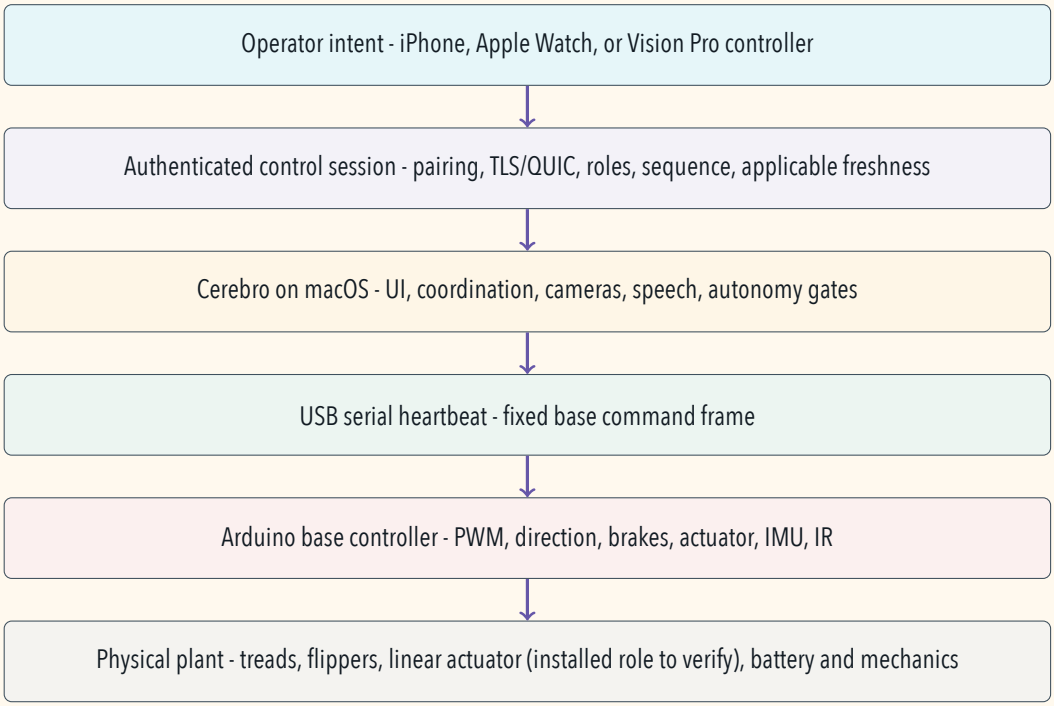
Follow the control stack

MISSION 1

One goal, many programs

Imagine an operator nudges a joystick forward. ROB's controllers encode that gesture in different ways: the current iPhone path places raw joystick coordinates in its legacy snapshot, while the Vision Pro controller derives normalized linear and angular intent. A secure network carries the intent to Cerebro on the Mac mini. Cerebro decides whether that controller owns motion authority, translates the values into the historical base frame, and refreshes the Arduino over USB. The Arduino sets pins. Motor electronics drive the treads.

The USB device name can change when a cable moves to another hub. Current Cerebro therefore scans only USB callout ports, resets each candidate, and listens for the existing Base sketch's exact line, `BEGIN BASE STARTUP SEQUENCE`. It sends no serial command bytes and requires no Arduino firmware flash. The refresh control closes the old Base connection and runs discovery again. The startup label selects the intended firmware role; it is not authentication and does not prove the harness is safe.

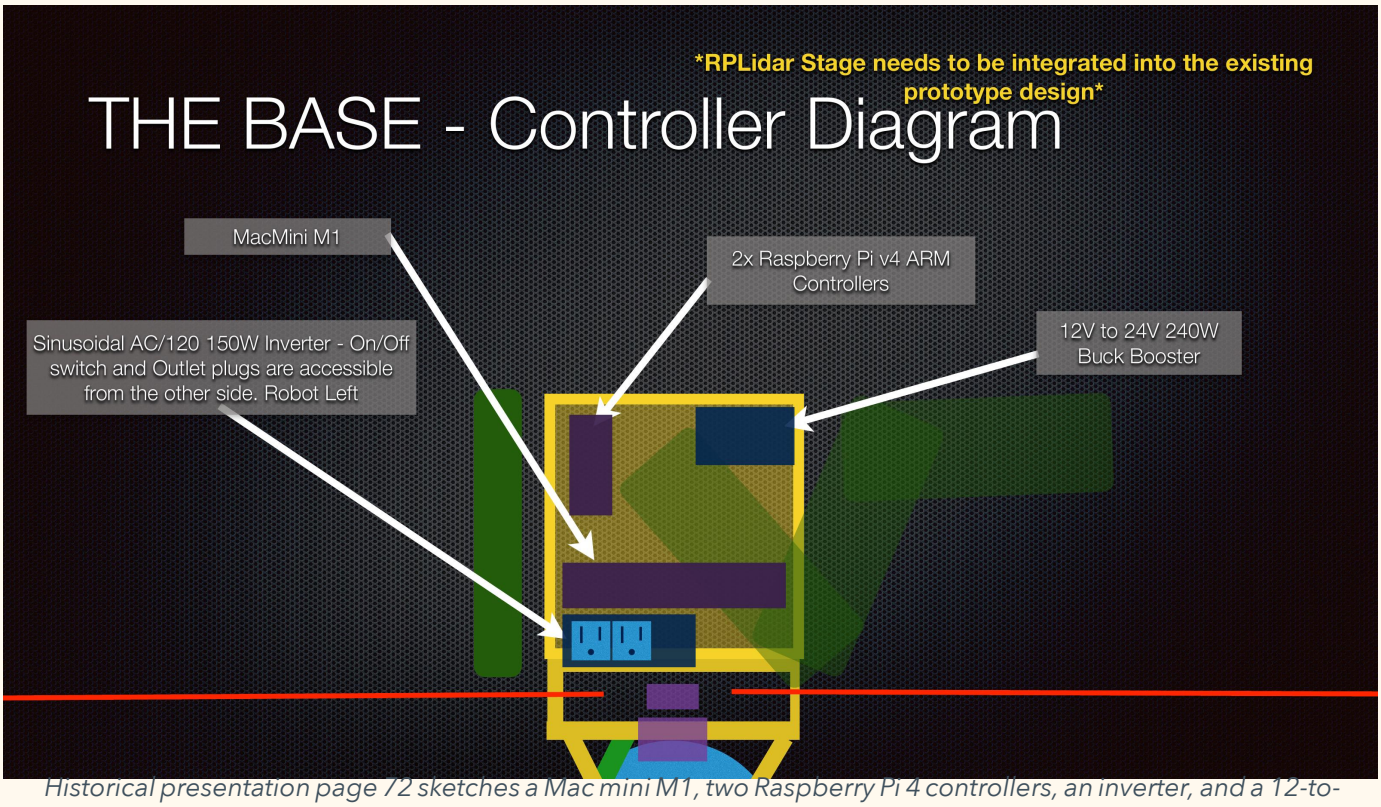


This architecture separates concerns:

- **Interface**: What does the human mean?
- **Transport**: Which authenticated device sent the message, and is it fresh?
- **Coordination**: Who currently controls the robot, and what behavior is allowed?
- **Hardware adapter**: How does a controller command become ROB's legacy seven-field serial frame?
- **Firmware**: Which output pins and controller bytes represent the command?
- **Physical plant**: What motion actually occurs?

COMPUTER SCIENCE CONNECTION

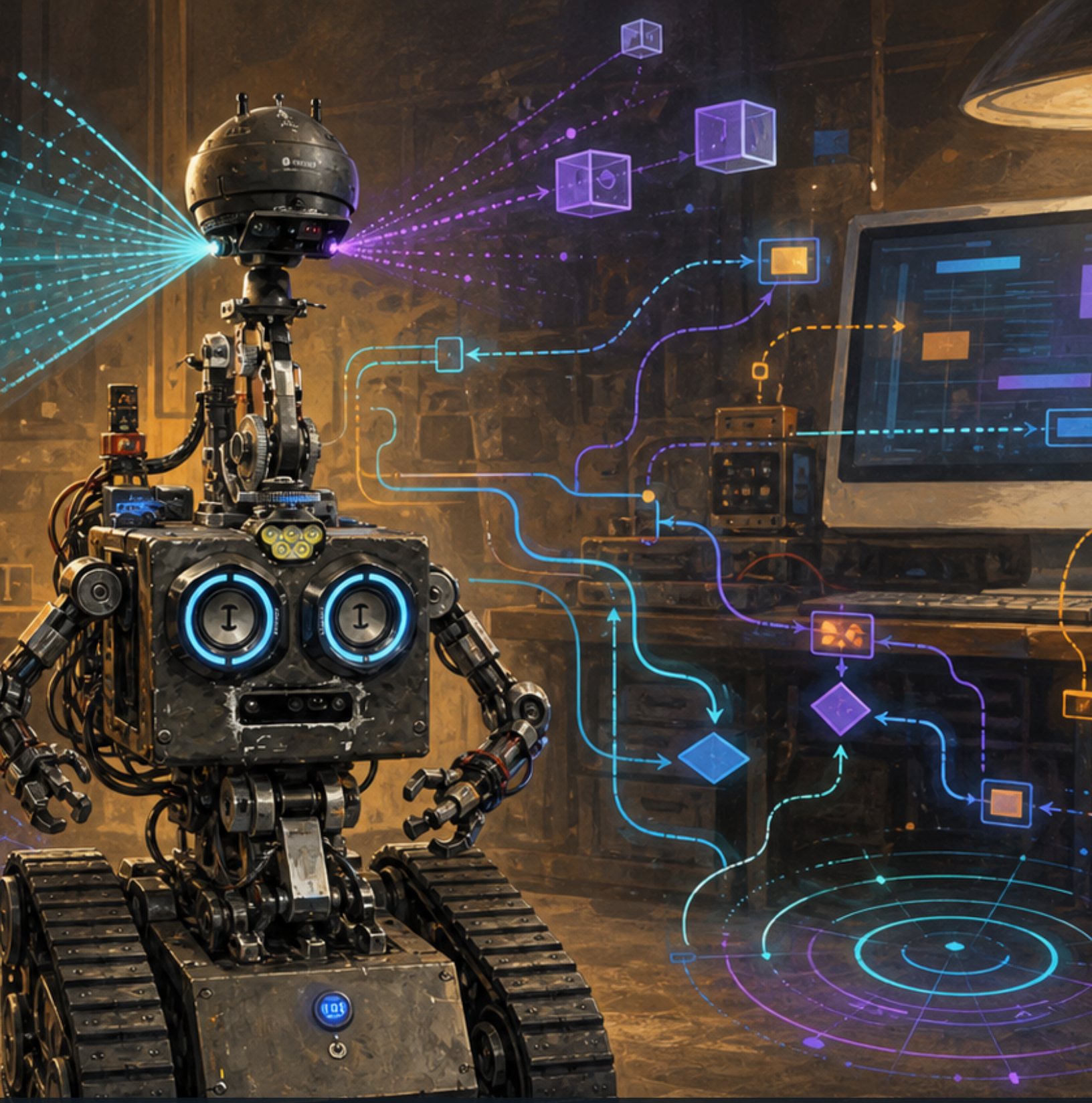
An **interface** is a promise between components. It should say valid inputs, outputs, timing, errors, and ownership. Internal code can change while the promise stays stable. That is how large systems remain understandable.



Historical presentation page 72 sketches a Mac mini M1, two Raspberry Pi 4 controllers, an inverter, and a 12-to-24 V converter. It records an architecture idea; the present hardware arrangement needs builder confirmation.

BUILDER INPUT NEEDED

Confirm the current onboard computer inventory, operating systems, power converters, physical ports, USB device assignments, network topology, and which Raspberry Pi roles remain active.



MISSION 2

Cerebro coordinates the body

MISSION 2

The Mac mini as a robot computer

SOURCE TRAIL — ANALYZING NOW

File: `Cerebro/Cerebro/ROBMainViewController.mm`
Why it is here: This central coordinator joins controller intent, robot state, helpers, autonomy, and operator-visible behavior. Follow its referenced classes rather than treating one large file as the entire system.

Cerebro is the macOS controller and operator interface. Its current source contains several distinct responsibilities:

- windows and controls for a human operator;
- the present Base Arduino serial connection, plus legacy Cerebro paths named for retired Torso and Head Arduinos and servo hardware;
- authenticated sessions from controllers and a role-limited lidar publisher;
- camera capture, RGB-D helper supervision, Vision processing, and video service;
- local speech recognition and speech synthesis;
- bounded autonomy coordination and optional Gemini integration;
- arm scripts and reusable keyframe animation data.

The project mixes Swift with Objective-C and Objective-C++. That is not automatically bad. Mature systems often cross language generations. A narrow adapter can let new typed code cooperate with a historical interface while migration happens safely.

A fanless mini computer with labeled USB and serial connections shows the practical side of software architecture.

Original illustration: abstract paths represent control, perception, and state without exposing real logs, accounts, or network details.

Event loops and state

A graphical Mac app reacts to events: a button click, a network message, a camera frame, a timer, or serial data. Long work cannot freeze the main event loop. Camera processing drops stale frames rather than building an unlimited backlog. A Python DepthAI helper is supervised as a separate process so a camera disconnect or SDK failure does not crash robot control.

COMPUTER SCIENCE CONNECTION

This is **fault isolation**. If one optional subsystem fails, the system reports its own unavailable state while unrelated safety and control paths continue. "Everything in one giant loop" is simple only until one slow task blocks every other job.

Mac message lab: build, inspect, do not transmit

The real Mac adapter is Objective-C, but a short Python exercise can reveal the same message shape without opening a serial port. Save this on a Mac and run it in Terminal. It only prints one neutral classroom frame.

Listing 2.1: *A print-only Mac exercise; it does not connect to ROB.*

```
def field(value):
    if not -9999 <= value <= 9999:
        raise ValueError("outside signed five-character range")
    return f"{value:+05d}"

values = [0, 0, 0, 0, 0, 0, 0]
frame = "~" + ",".join(field(value) for value in values)

assert len(frame) == 42
print(frame)
```

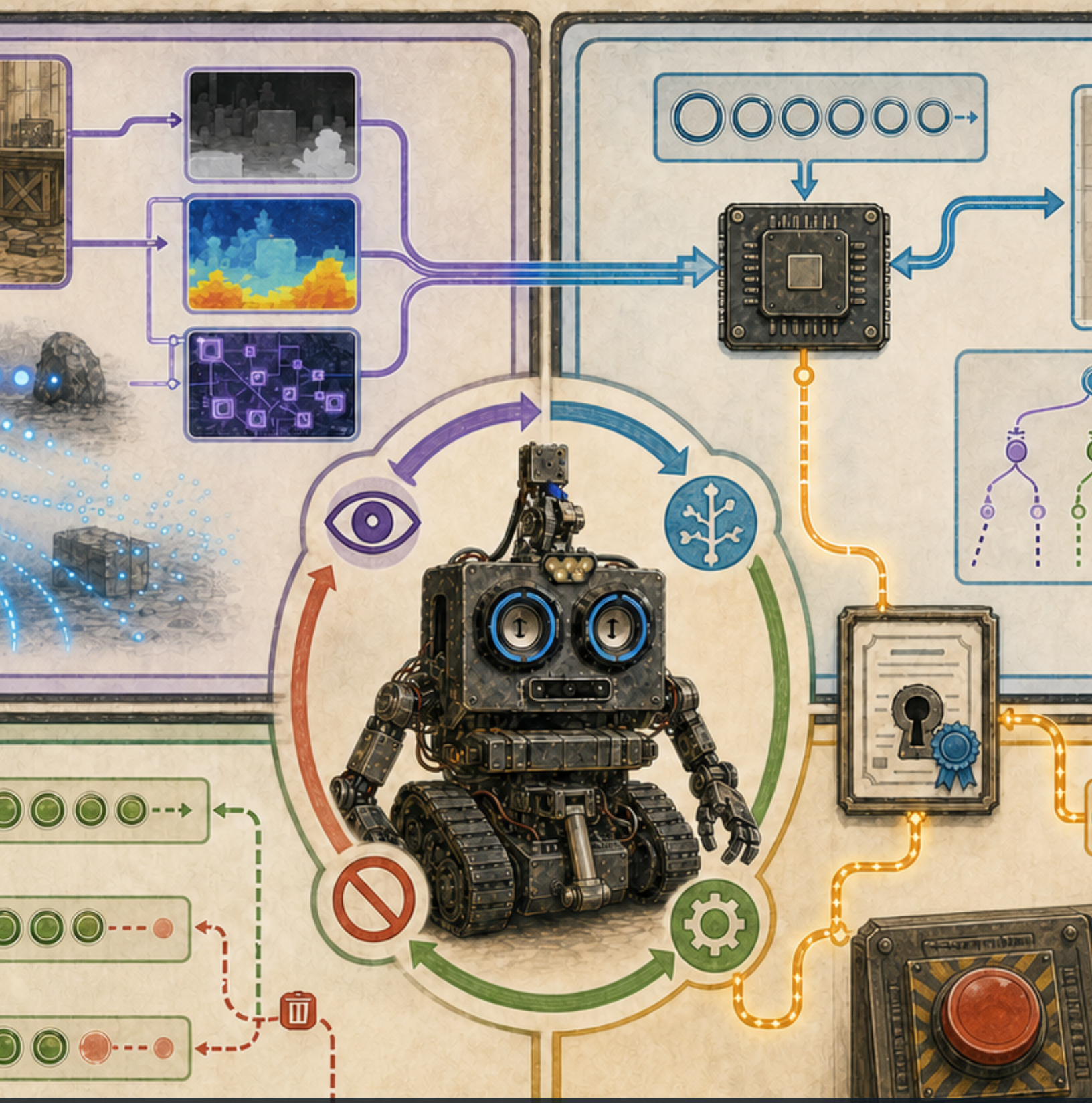
The companion file `tools/mac_serial_frame_lab.py` also parses a frame and labels all seven values. A hardware version would need an approved serial device, exact baud configuration, complete input validation, a physical stop path, and supervised tests; those capabilities are intentionally absent from the lab.

CHANGE ONE THING

Run the offline lab, change one tread value from 0 to 100, and compare the two strings. Predict which five characters will change before running it. Then deliberately remove a comma and ask the companion parser to reject the frame.

STATE-MACHINE CARDS

Make cards labeled Disconnected, Connecting, Authenticated, Armed, Stopped, and Faulted. Draw only the allowed transitions. Does reconnecting automatically reset a stop? It should not. Add the event required for each arrow.



MISSION 3

Many controllers, one authority

MISSION 3

Phone, Watch, and Vision Pro

SOURCE TRAIL — ANALYZING NOW

File: `ROBController/Consciousness/ConsciousViewController.mm`
Why it is here: This is the starting point for the handheld controller UI. Compare its shared message types with the corresponding decoder in Cerebro and the SwiftUI spatial controller under `ROBControllerVision`.

ROB has several ways to express operator intent, but only one component should own motion at a time.

DEVICE	ROLE	IMPORTANT BOUNDARY
iPhone / iPad	Joysticks, speed, flipper, actuator, autonomy and AI-action UI.	Holds its own paired controller identity and sends the established controller payload.
Apple Watch	Voice and joystick intent relayed through the phone.	Does not browse for ROB or store robot credentials; stale watch motion stops at the relay.
Vision Pro	SwiftUI control experience, simulator, normalized commands, and optional live video.	Uses a short input lease and a separate authenticated video connection.

Normalized motion usually means values between -1 and +1. That keeps a user interface independent of physical top speed. The robot later converts the normalized request into its configured limits.

One readable differential-drive mix is:

$$L = \text{clamp}(F + T, -1, 1), \qquad R = \text{clamp}(F - T, -1, 1)$$

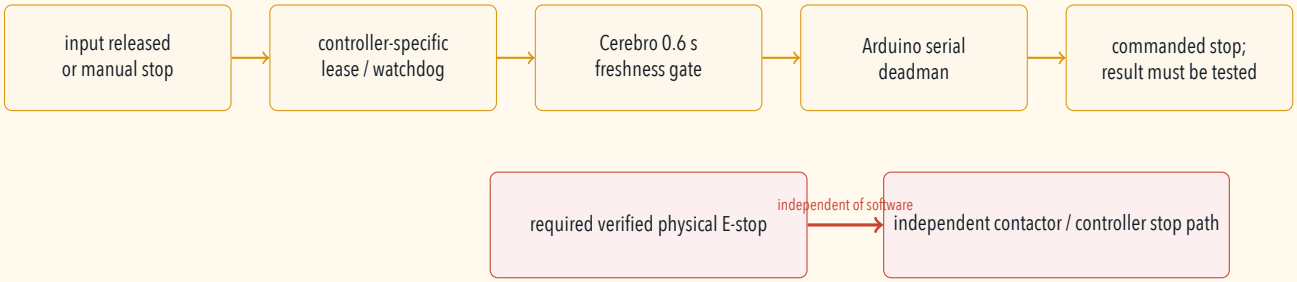
where F is forward intent and T is turn intent. A speed cap multiplies both outputs before they reach the robot.

JOYSTICK GRAPH

Draw a coordinate square from -1 to +1 on each axis. Choose five (F, T) points. Calculate L and R, clamp them, and predict the motion. Then reverse the sign convention and notice why written coordinate definitions matter.

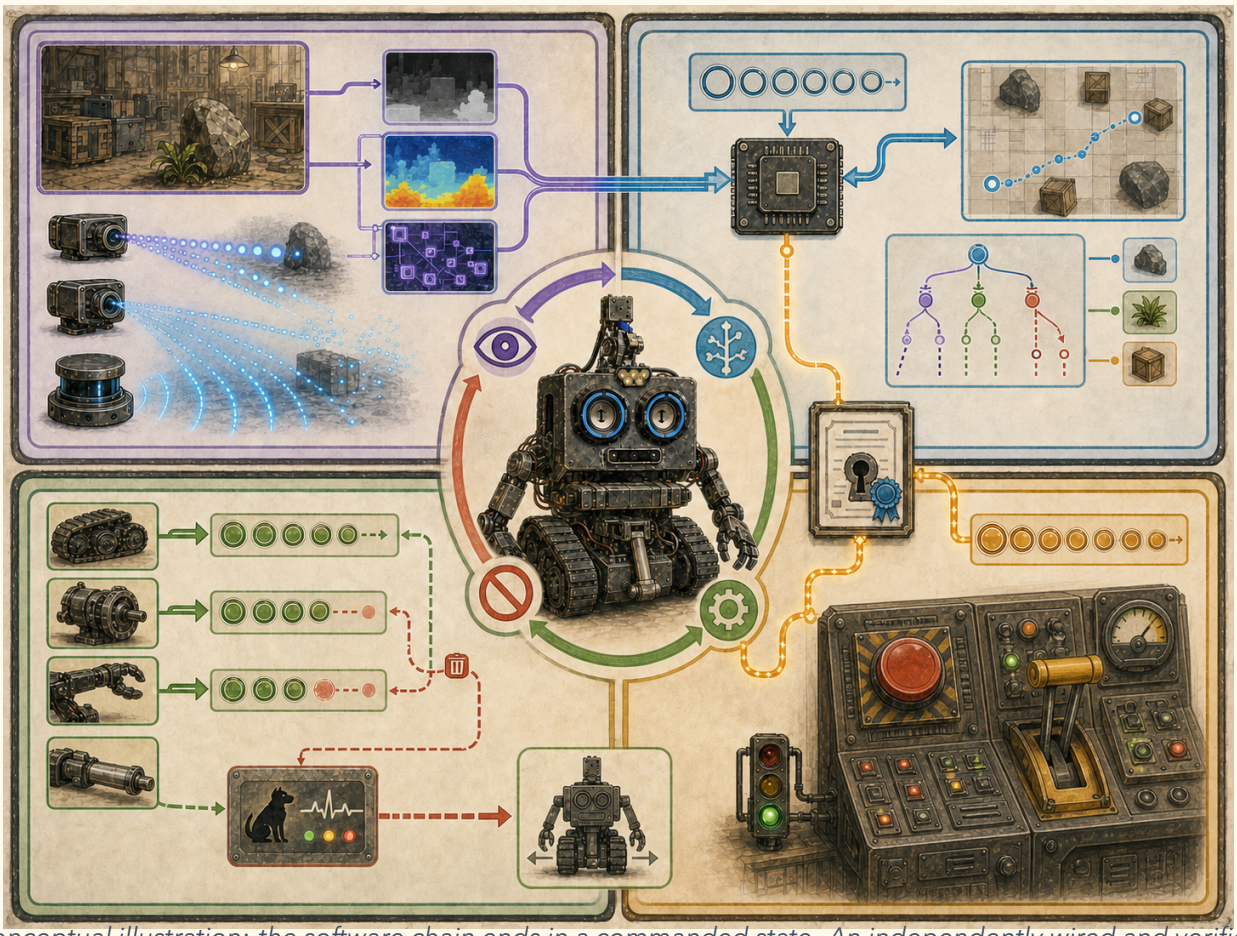
Freshness is part of the command

Current controller layers use short timing limits: the Vision control model uses a 250 ms lease, the Watch relay uses roughly a 450 ms watchdog, and Cerebro expires a stale control snapshot at about 600 ms. These values belong to different layers and do not prove the Arduino's loop-count heartbeat duration.

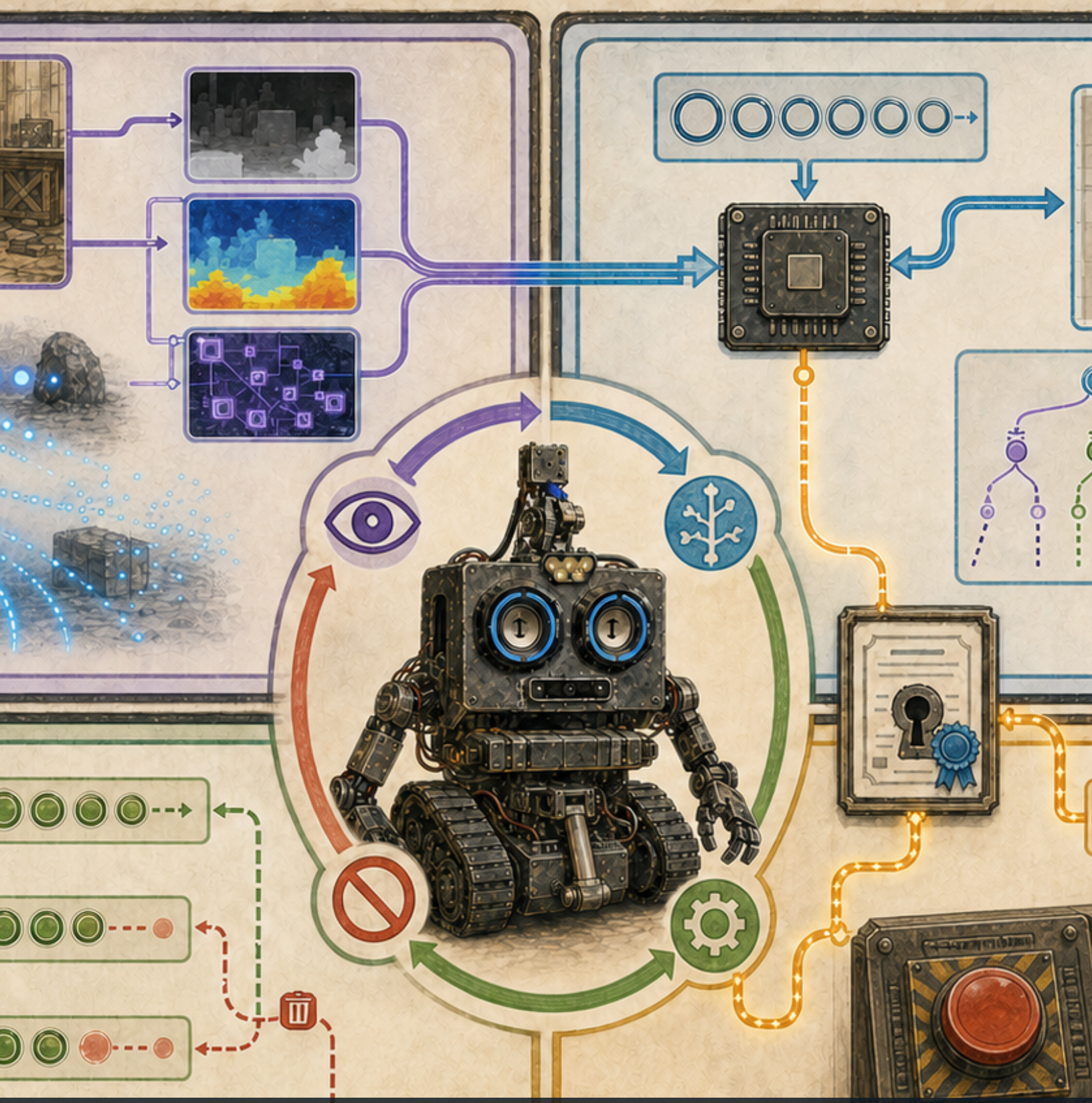


FROM ROB'S BUILD LOG

Cerebro sends one neutral/braked base frame when controller snapshots expire, then stops refreshing USB so the Arduino's independent heartbeat also expires. The current Arduino timeout command uses zero speeds with brake fields that map to released brakes. The physical coast/hold result must be measured; do not call it "braked" at the Arduino layer.



Conceptual illustration: the software chain ends in a commanded state. An independently wired and verified physical stop must still interrupt motion authority when software, communications, or assumptions fail.



MISSION 4

Discovery is not trust

MISSION 4

Pairing and secure robot messages

SOURCE TRAIL — ANALYZING NOW

File: `ROBController/Consciousness/AutoNetClient/AutoNetDataTransferProtocol.swift`
Why it is here: This client-side wire-contract implementation is the file being analyzed for identity, framing, authentication, sequencing, and freshness. Compare it with `Cerebro/Cerebro/AutoNet/AutoNetShared/AutoNetDataTransferProtocol.swift`.

ROBControl v2 uses a reliable QUIC stream over UDP with TLS 1.3. Cerebro advertises a local Bonjour service named `_robctl._udp` and uses the application protocol name `robctl/2`. Those names help a controller find a candidate robot, but discovery alone does not prove identity.

A simplified pairing story is:

1. Cerebro creates a code for one named device and one narrow role.
2. The device stores the robot ID, its own credential ID and secret, and the exact server certificate fingerprint in Keychain.
3. On connection, TLS protects the link and the device pins the expected certificate.
4. Both sides prove knowledge of the pairing secret using a fresh challenge.
5. Each layer performs the checks that apply to its data: transport framing checks size, role, session, and sequence; timestamped sensor messages add capture-time freshness; controller freshness uses Cerebro's local receipt time; and the legacy application payload still needs its own parsing and validation.

COMPUTER SCIENCE CONNECTION

Authentication asks who the peer is. **Authorization** asks what that peer may do. ROB's lidar publisher may send typed sensor data but cannot request motion. A controller may request allowed control messages but cannot become a lidar publisher just by changing a word in its own payload.

NEVER PRINT REAL PAIRING MATERIAL

Book screenshots and Maker Faire displays must not reveal enrollment codes, API keys, Wi-Fi passwords, SSH credentials, private keys, device secrets, precise location data, or private console logs. A server certificate fingerprint is a public identity pin, not a secret; it may still be redacted from screenshots as operational metadata. Give every device a unique secret so one device can be revoked without disabling another.
The current iPhone controller requests Always location access and includes live latitude and longitude in each active legacy snapshot. Motion control does not inherently need precise location. Treat this as current privacy debt: remove it, minimize it, or document a specific need and consent policy before a public demonstration.

LOCKED MAILBOX GAME

Give three students cards: Operator, Lidar, and Guest. Write allowed message types on role cards. For each pretend packet, check identity first, then role, then sequence number, then freshness. A correctly signed Lidar packet still may not contain a drive command.

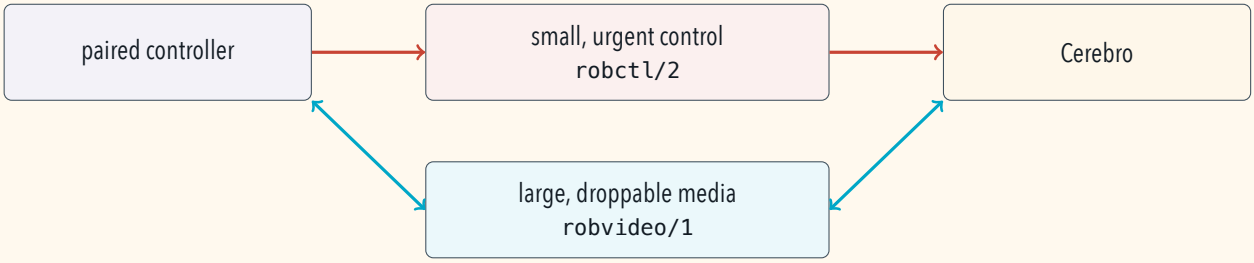
MISSION 5

Keep video away from stop traffic

MISSION 5

Control and camera data take separate roads

Live video is much larger than a joystick command. If both share one congested queue, a delayed camera frame could sit in front of a stop message. Cerebro therefore advertises a separate authenticated `_robvideo._udp` service using `robvideo/1`. The initial profile sends H.264 video on its own QUIC connection.



The video connection still depends on the exact live control session and the same authenticated controller. If control disconnects, the associated video session closes. If video fails, control can remain available.

COMPUTER SCIENCE CONNECTION

This is a **quality-of-service** decision expressed through architecture. Urgent control is small and bounded. Media may drop old frames when the sender falls behind. The freshest useful image is usually better than a growing pile of late images.

MISSION 6

Color, depth, and laser points

MISSION 6

How ROB sees space

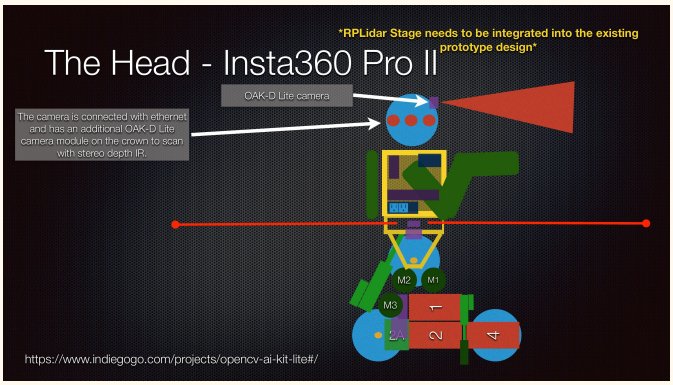
SOURCE TRAIL — ANALYZING NOW

File: `Cerebro/Cerebro/CameraManager.swift`
Why it is here: This file owns a bounded camera/depth delivery path. Lidar acquisition lives separately under `M2M1-RPLIDAR-iOS-MacOS-Catalyst-/RPLidar`; the chapter names the source when it changes sensor.

ROB's camera path can produce aligned color and depth. A color pixel describes brightness and color channels. A depth pixel describes estimated distance, currently carried as an unsigned 16-bit number of millimeters; zero means invalid. Alignment lets software ask, "What distance belongs to this color pixel?"

A supervised Python DepthAI helper owns the OAK-D device and sends synchronized data to Cerebro through a user-only local Unix socket. RGB enters the camera/Vision path. Depth travels beside it in a frame set. If the helper or device fails, Cerebro reports camera state and retries with bounded backoff instead of terminating robot control.

An OAK-D stereo depth camera has multiple optical elements because depth requires geometric information beyond one ordinary image.



Historical presentation page 74 combines the Insta360 head concept with an OAK-D Lite crown camera. Confirm the current head configuration before final captions.

Lidar uses polar samples

RPLidar reports distances at angles around the robot. A point (r, θ) can be converted into a flat Cartesian point:

$$x = r\cos\theta, \qquad y = r\sin\theta.$$

The companion app publishes typed, bounded scans and separate occupancy maps. Its current scan path first keeps only valid points closer than 1 m and within about ± 0.50 radians of the robot's front or back. The

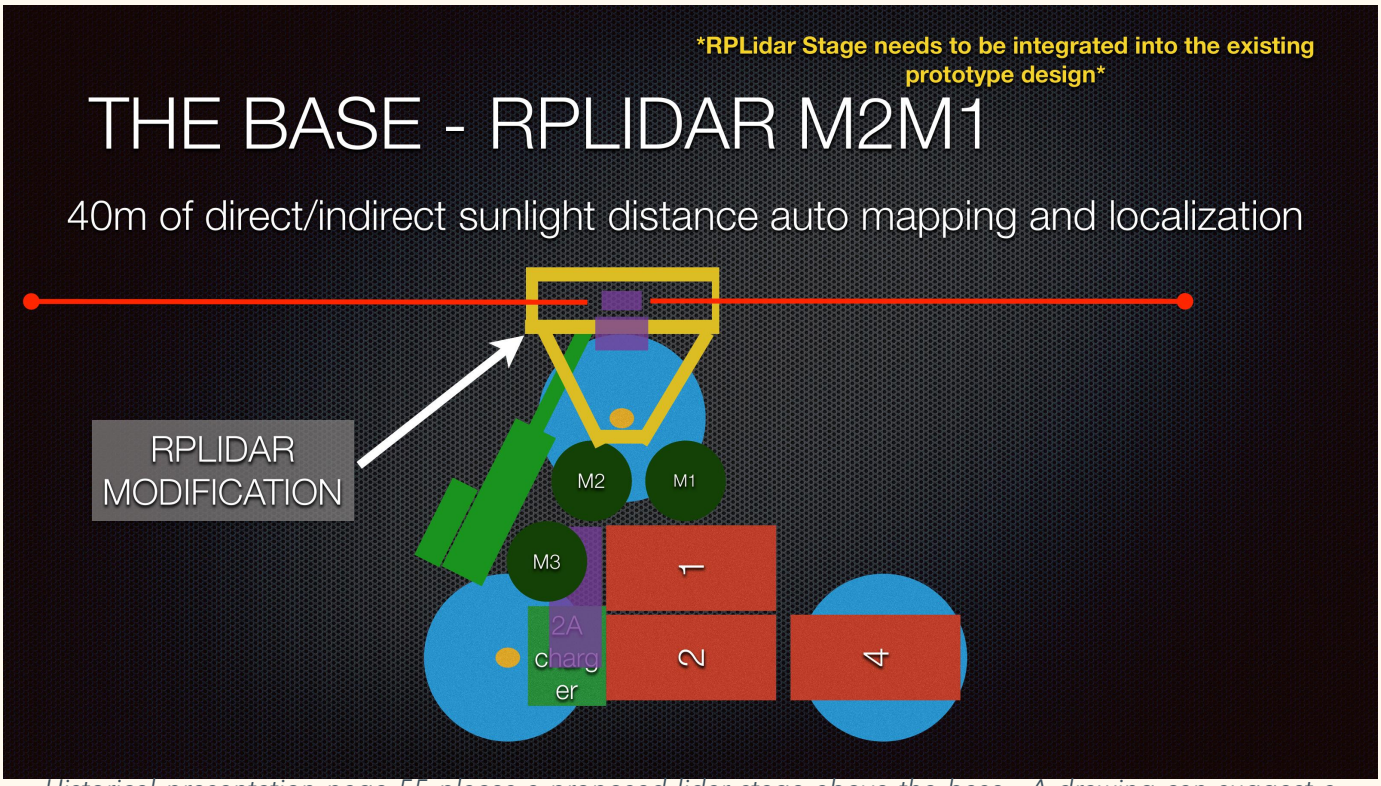
published scan is therefore not a full 360-degree view. Current local autonomy uses these fresh filtered samples, not the map, to choose cautious social-roam behavior. A stale sensor feed stops motion and leaves the autonomy session active in a waiting-for-lidar state.

DRAW A TINY LIDAR MAP

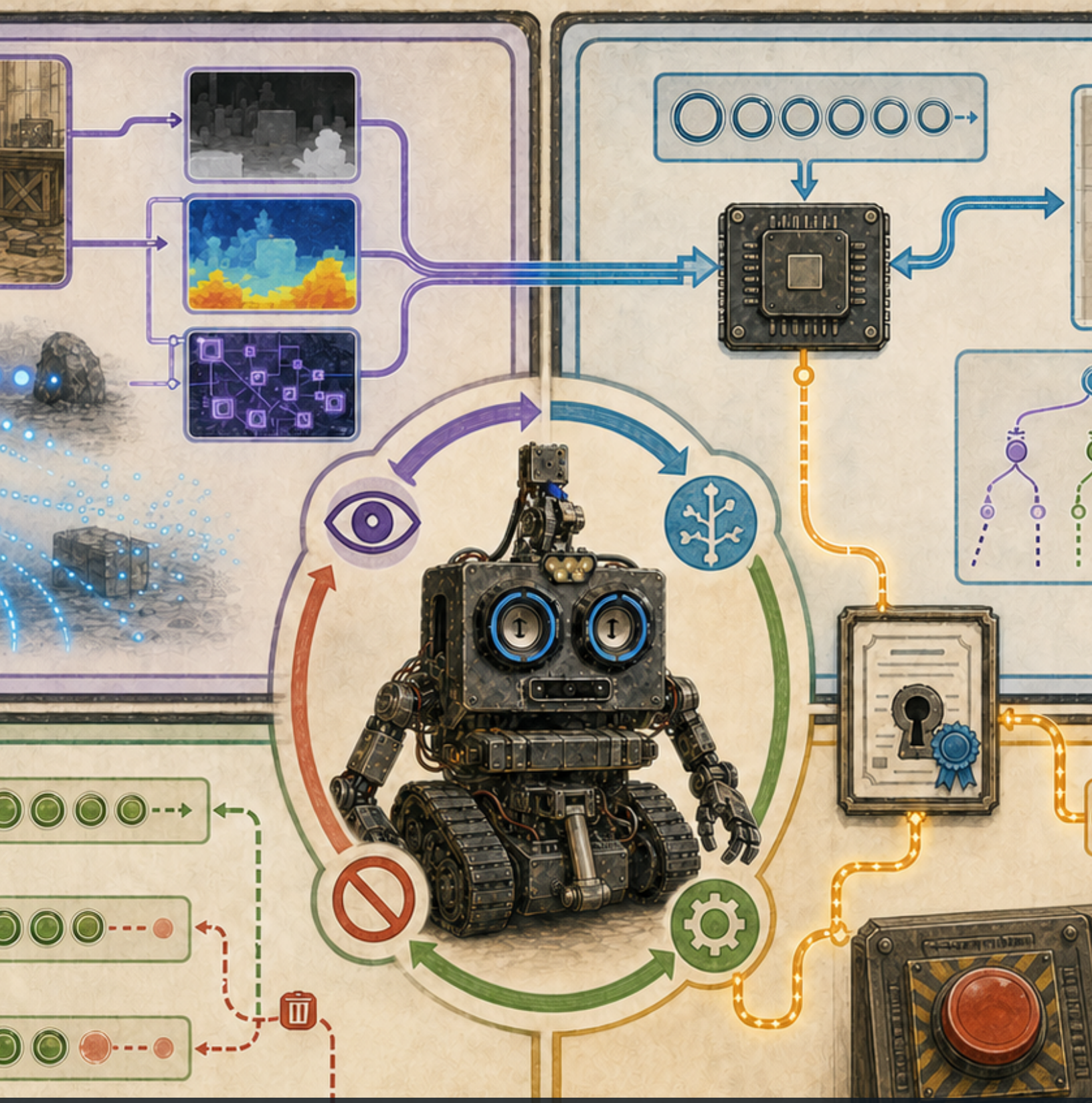
Plot these polar points on graph paper: $(4, 0°)$, $(4, 90°)$, $(4, 180°)$, and $(4, 270°)$. Convert each to (x, y). Add noisy points and decide how many samples you need before calling a region occupied.

ADVANCED CHANNEL

Coordinate frames must be named: sensor frame, robot frame, map frame, camera frame, and arm frame. A point is not useful for grasping until calibrated transforms connect those frames. The repository does not yet contain the complete camera-to-arm calibration, inverse kinematics, collision checking, or verified grasp executor needed for autonomous pickup.



Historical presentation page 55 places a proposed lidar stage above the base. A drawing can suggest a coordinate frame, but the installed transform must be measured.



MISSION 7

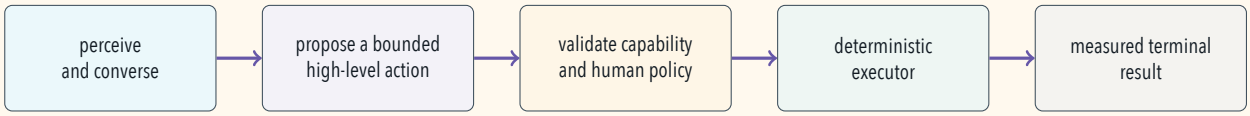
Speech and AI stay inside boundaries

MISSION 7

Helpful intelligence, human authority

Cerebro can recognize speech locally and speak responses through its existing voice path. An optional, explicitly enabled Gemini integration can receive microphone audio and sampled camera frames. That changes the privacy boundary because data may leave the robot for a cloud service.

The safer action pattern below is a *target architecture*. It shows the structure every physical action should eventually follow; it is not evidence that every current tool has an executor.



The language model does not receive a tool for arbitrary raw serial bytes. The current schema names five proposals: `look_at`, `play_gesture`, `request_pick`, `navigate_relative`, and `stop_motion`. Only `stop_motion` has an immediate local executor, and its result is deliberately partial: Cerebro can software-stop the base, but arm hold remains unverified. The other four actions do not yet have deterministic executors. Outside autonomy they may reach a controller approval console, but approval records a human decision and does not actuate hardware; during active autonomy they fail as unavailable.

ACTION	CURRENT STATUS	HONEST RESULT
`stop_motion`	Partial local executor	Preempts local coordinators, sends one neutral/braked base frame, and drops the base heartbeat; arm hold is unverified.
`look_at`	Schema only	No deterministic executor; approval alone does not move the neck.
`play_gesture`	Schema only	No deterministic executor; approval alone does not play a physical gesture.
`navigate_relative`	Schema only	No deterministic executor; it is not the social-roam controller.
`request_pick`	Planned / unavailable	Rejected until calibration, planning, feedback, and a verified grasp executor exist.

FROM ROB'S BUILD LOG

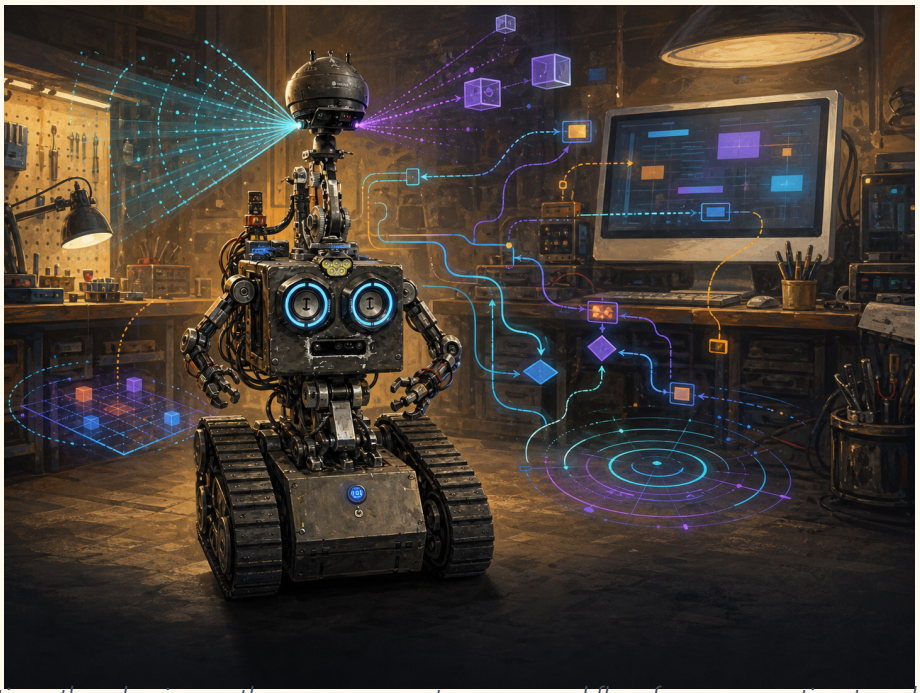
In the inspected snapshot, general picking remains unavailable. The source explicitly lacks calibrated camera-to-arm transforms, complete neck-axis mapping, joint feedback, inverse kinematics, collision checking, and a verified grasp executor. The controller's AI approval console records a decision but does not by itself execute an arm action.

PUBLIC CAMERA AND MICROPHONE NOTICE

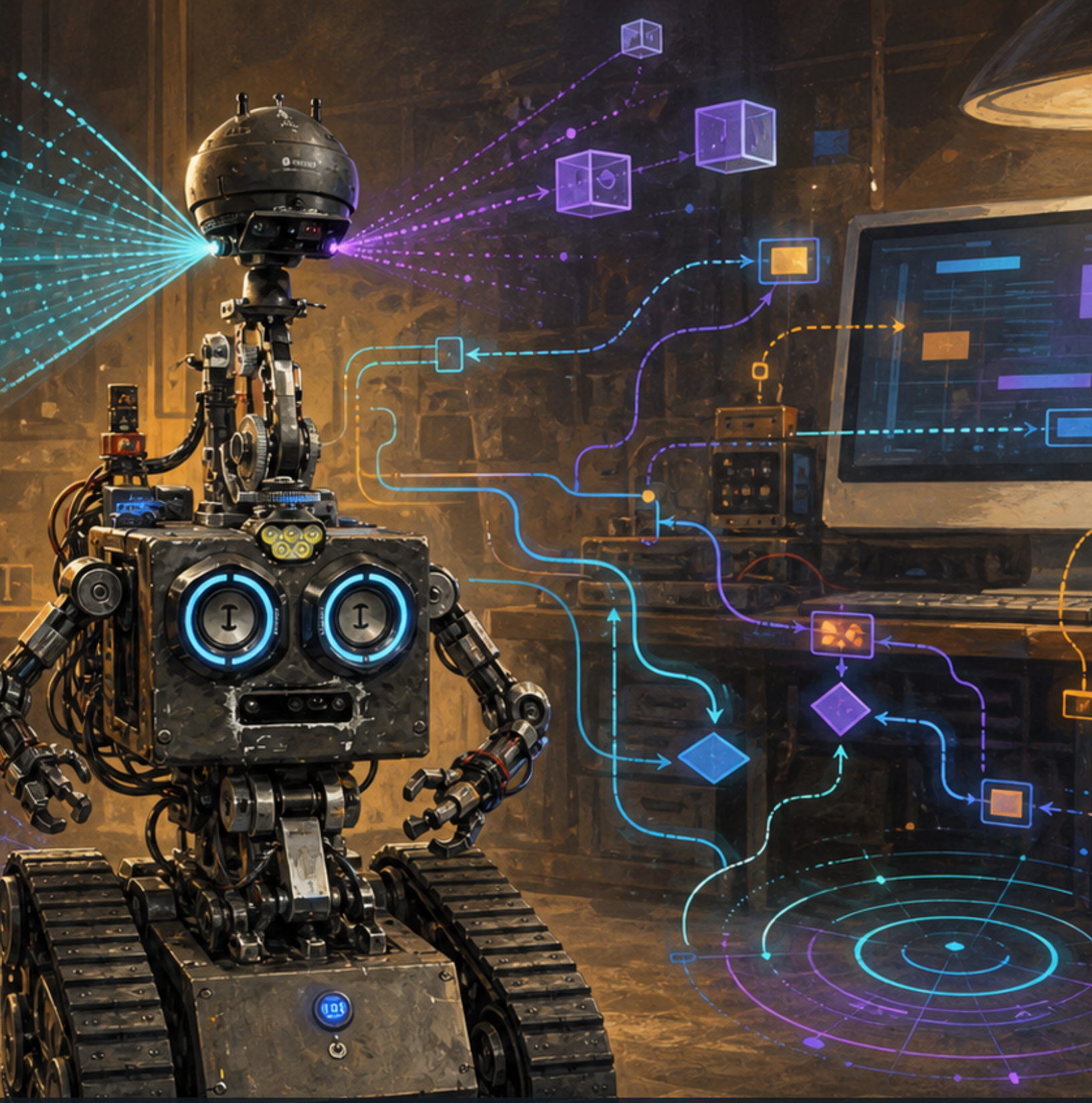
A Maker Faire demonstration should clearly announce when cameras, microphones, recording, or cloud AI are active; provide a visible activity indicator and rapid disable control; avoid collecting unnecessary location or identity data; and obtain appropriate consent, especially around children. The entire Gemini integration defaults off. Once it is enabled, however, the current configuration defaults microphone-audio and camera-frame streaming on unless each is separately disabled; the robot-action tool remains separately off by default. Verify every switch rather than relying on one "AI enabled" label.

HUMAN-IN-CHARGE ROLE PLAY

One student is the model, one the validator, one the operator, and one the simulator. The model may propose only named actions. The validator rejects unsupported or unsafe parameters. The operator approves or denies. The simulator returns a measured result. Discuss why "the model sounded confident" is not a validation rule.



Original illustration: the glowing pathways represent a proposed flow from perception to validation to deterministic execution. They are not a screenshot or proof that every action exists.



MISSION 8

Simulation makes mistakes inexpensive

MISSION 8

Tests, logs, and reproducible missions

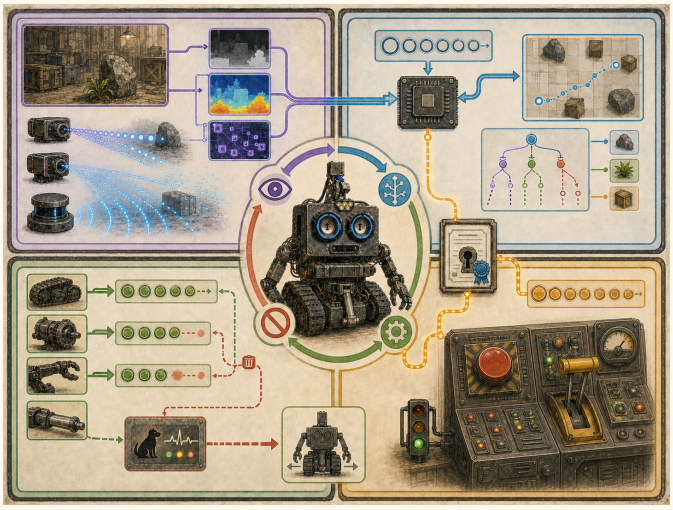
ROBControllerVision includes a deterministic simulator and a normalized control model. That is a powerful pattern: the same user interface can exercise a simulated endpoint before a physical adapter is selected.

A useful test names:

- initial state;
- exact input and timestamp;
- expected output or rejection;
- deadline;
- cleanup behavior;
- evidence retained when it fails.

FAILURE TO INJECT	EXPECTED SAFE OBSERVATION
Joystick release	Zero intent is sent promptly.
Watch link disappears	Phone relay stops watch motion after its freshness deadline.
Controller snapshots stop	Cerebro neutralizes once and lets the USB heartbeat expire.
Malformed ROBControl frame	Transport rejects it before application dispatch.
Malformed legacy controller payload	The application decoder must reject it; authenticated transport alone does not make payload contents valid.
Video sender falls behind	Old video frames drop; control queue remains independent.
Depth helper exits	Camera reports unavailable and retries; robot control remains alive.
Lidar becomes stale	Autonomy stops and waits rather than inventing space.
Unsupported AI action	A clear unavailable result returns; no motion is faked.

Original illustration: software pathways are explanatory symbols, not a current-system screenshot.



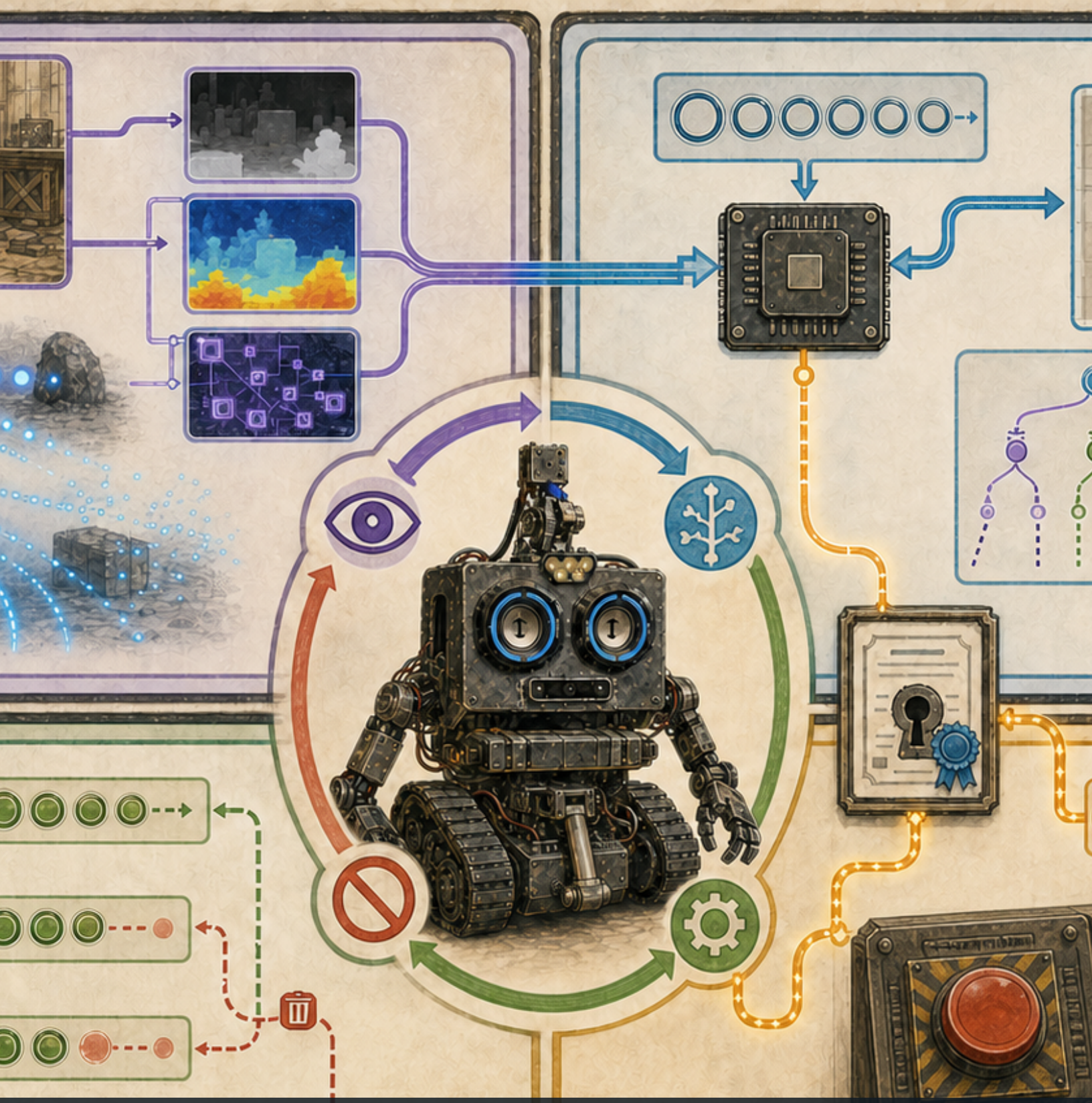
Conceptual illustration: the test ladder leads from bounded simulation toward reviewed physical tests; a successful display alone does not prove safe behavior.

WRITE A TEST TABLE

Choose one simulated command. Create columns for sequence number, issue time, lease, left value, right value, and expected state. Add stale, duplicate, NaN, and out-of-range cases. A good test includes messages that must be rejected.

BUILDER INPUT NEEDED

Before publication, add a tagged source snapshot, screenshots with all secrets and location data removed, a verified component version table, actual on-robot latency measurements, failure-injection results, and a diagram of current rather than proposed compute hardware.



DEEP LAB

Trust, time, and authority

MISSION 9

A robot never receives ``just a command"

A useful control message carries meaning in context: identity, role, session, sequence, issue time, lease, units, bounds, and intended recipient. Transport reliability can deliver bytes in order, yet the application must still decide whether those bytes are current, authorized, well formed, and safe to apply.

Discovery, authentication, and authorization

Discovery answers “What services are nearby?” Authentication answers “Can this peer prove the installed identity?” Authorization answers “May this identity publish lidar, operate controls, or view video?” A certificate fingerprint can bind a controller to one robot, while a fresh challenge and keyed proof resist copied or replayed conversations. None of these checks replaces command bounds or a physical stop.

THE FOUR GATES

Make four paper gates labeled Find, Prove, Permit, and Validate. Give each student a message card with a service name, device identity, role, sequence, age, and value. A card advances only when each gate’s rule passes. Try a real identity with the wrong role, an old sequence, and an out-of-range value. Explain why one successful gate cannot answer the others.

Freshness is a continuously expiring claim

Suppose a command was valid when issued. Network delay, a paused app, or a lost controller can make it dangerous later. A monotonic clock measures elapsed time without being confused by wall-clock corrections. A lease says how long intent remains usable. Sequence numbers reject duplicates and reordering. A watchdog chooses a defined response when renewal stops.

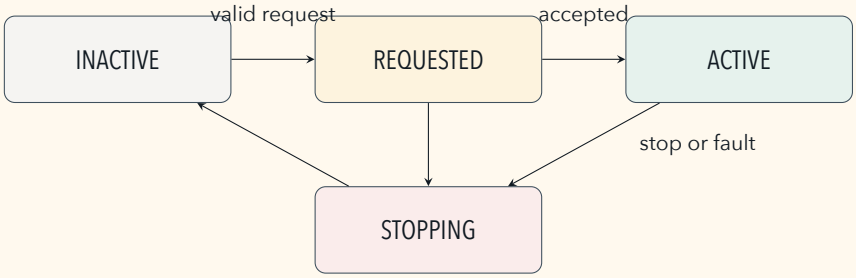
LATENCY HAS A BUDGET

End-to-end response includes sensing, sampling, encoding, queueing, network transit, validation, scheduling, actuation, and physical response. Measure distributions, not only one best case. Report typical behavior, high-percentile delay, worst observed cases, test conditions, and what happens after the deadline.

MISSION 10

Autonomy is a bounded state machine

"Autonomous" should not mean "unlimited." A bounded session has an owner, profile, start condition, radius or workspace, speed limit, required sensors, freshness rules, timeout, stop transitions, and evidence. Manual control, stale perception, invalid calibration, internal faults, and operator stop can all end authority.



Perception is not a permission slip

A camera or lidar measurement may support a world model, but it cannot prove the path is safe outside its field of view, range, calibration, timing, and tested conditions. Planning proposes a path through a model. Validation checks policy and bounds. A deterministic executor converts an accepted action into controlled outputs. Feedback tests whether reality agrees.

OFFLINE MISSION SIMULATOR

Draw a grid with a start, goal, obstacles, and unknown squares. One student reveals a sensor-limited neighborhood. One proposes a one-step move. One validates boundary, freshness, and collision rules. One records the new state. Add a stale sensor card and a manual-control card; both must end autonomous authority. No physical robot is used.

AI PROPOSES; VERIFIED SOFTWARE DISPOSES

Natural-language models can help interpret or propose, but physical actions need typed schemas, bounded parameters, current state, policy, human authority where required, deterministic execution, cancellation, telemetry, and honest unavailable results. Fluency is not evidence of calibration, reachability, collision clearance, or completion.

Field words

ABSTRACTION A simpler interface hiding internal detail.

AUTHENTICATION Proof of identity.

AUTHORIZATION Rules about allowed actions.

BACKPRESSURE What a system does when data arrives too quickly.

COORDINATE FRAME A named origin and axes for measurements.

LEASE Permission that expires unless renewed.

PROCESS A running program with its own resources.

PROTOCOL An agreement about messages and behavior.

SEQUENCE NUMBER A counter used to detect replay or reordering.

SIMULATION A model used to test behavior without the full physical system.

FINAL CHECK

Can you trace a joystick command to the Arduino? Can you explain why discovery does not prove trust? Can you explain why video uses a separate connection? Can you place an AI proposal behind a validator and human policy? You are ready for the complete field manual.

Source trail: Current 2026-08-02 source snapshot: Cerebro, ROBController, ROBControllerVision, and M2M1-RPLIDAR; their active docs; ROB base firmware; private gallery originals. Experimental and uncommitted Cerebro work is labeled as such.